



GitHub Copilot in Visual Studio

Agents that debug, profile, and test

Original by:

Nik Karpinsky

Mads Kristensen

Bangkok build by:

Sivaphong Niyomphanich



<https://github.com/pergiong/Microsoft-Build-Localhost-Bangkok-2026>

About Me



- Sivaphong Niyomphanich - Pong+
 - E25ZXQ
- SCB 10X
- AiAT (Artificial Intelligence Association of Thailand)
- 22 Nor 4x4 Association
- SuperAI Engineer SS#2 (Bronze Medal)



Professional developers

Quality gates and tenants

Code as an **asset** –
not an artifact

Before The Demo

Session Resources

Visual Studio blog

visualstudio.blog

Connect with the team

aka.ms/vshub

Session GitHub Repo

aka.ms/build26/brk207

Demo GitHub Repo

github.com/pergiong/Microsoft-Build-Localhost-Bangkok-2026

Agent Resources

Visual Studio Insiders

visualstudio.com/insiders

GitHub Copilot

github.com/features/copilot

Visual Studio 2026 Visual Studio 2022 v17.14+

GitHub Copilot Pro Plan ++

github.com/features/copilot/plans

GitHub Copilot debugger, profiler and test

(Agent available on GitHub Copilot Pro ^^)

Agent	Purpose	Key Features
@debugger	Error diagnosis	Reads call stacks, variable states, reproduces bugs, sets tracepoints & conditional breakpoints.
@profiler	Performance optimization	Connects to profiling tools, identifies bottlenecks, suggests targeted fixes.
@test	Unit test generation	Creates framework-specific tests, avoids boilerplate rejected by CI.



Copilot Agents Cheat Sheet



@debugger

Debug Your Code

Ctrl + /

➤➤➤ *Fix this bug*



@profiler

Analyze Performance

Ctrl + /

➤➤➤ *Optimize this loop*



@test

Generate Unit Tests

Ctrl + /

➤➤➤ *Create test cases*



Enable Copilot Agents

ON



Press Ctrl + /



Pick Your Agent



Debugger



Profiler



Test

GitHub Copilot debugger, profiler and test

Tools → Options → GitHub → Copilot → Enable Copilot Agents

Agent	Shortcut	Purpose	Example Command
@debugger	Ctrl + / → <i>Debugger</i>	Detects and fixes bugs instantly	“Find the cause of this exception.”
@profiler	Ctrl + / → <i>Profiler</i>	Analyzes performance bottlenecks	“Analyze performance of this loop.”
@test	Ctrl + / → <i>Test</i>	Generates complete unit test suites	“Generate unit tests for this class.”

But currently

Pro

For everyday coding with agents in GitHub Copilot.

\$10 USD

per user / month



New plan sign-ups are temporarily paused as we ensure a high-quality experience. We appreciate your patience. [Learn more](#)

Pro+

For more complex development with premium models.

\$39 USD

per user / month



New plan sign-ups are temporarily paused as we ensure a high-quality experience. Existing Student and Pro customers can [upgrade to Pro+](#). We appreciate your patience. [Learn more](#)

Max

For sustained, high-volume agent workflows with GitHub Copilot.

\$100 USD

per user / month



New plan sign-ups are temporarily paused as we ensure a high-quality experience. Existing Student, Pro, and Pro+ customers can [upgrade to Max](#). We appreciate your patience. [Learn more](#)

Demo

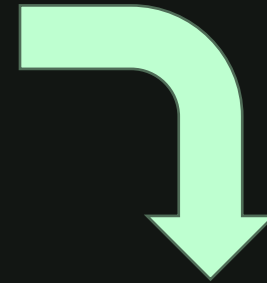
@debugger

“Copilot Debugger doesn’t just point out bugs — it crafts reproducible scenarios and delivers actionable fixes instantly.”

@debugger

```
Program.cs  data_processing.py  Calculator.cs
Miscellaneous Files  Program
1  // Program.cs
2  using System;
3
4  class Program
5  {
6      static void Main()
7      {
8          string? name = null;
9          Console.WriteLine("Hello " + name.ToUpper());
10     }
11 }
12
```

Why does this `code` throw an exception? | GPT-5 mini



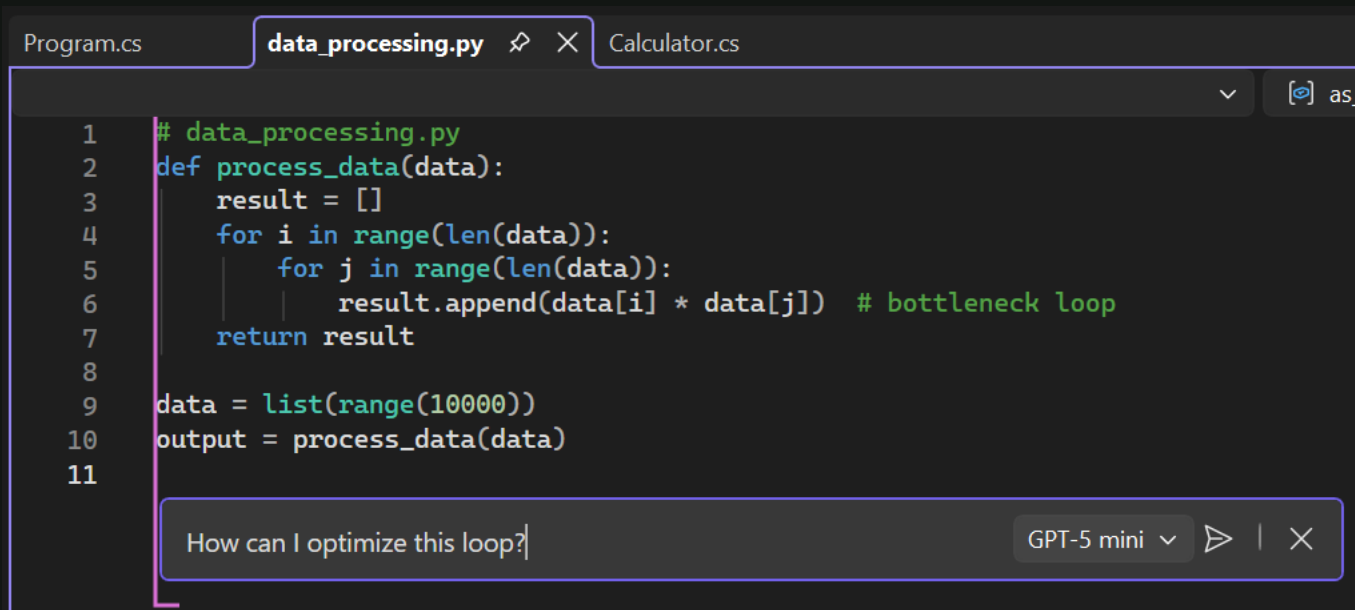
Prompt: “Why does this code throw an exception?”
Prompt (fix): “Fix this code so it doesn’t throw an exception.”
Prompt (explain): “Explain the root cause of this bug.”

```
Program.cs  data_processing.py  Calculator.cs
Miscellaneous Files  Program
1  // Program.cs
2  using System;
3
4  class Program
5  {
6      static void Main()
7      {
8          string? name = null;
9          Console.WriteLine("Hello " + name.ToUpper());
10         Console.WriteLine("Hello " + (name?.ToUpper() ?? "Guest"));
11         // TODO: Fix NullReferenceException
12         // NullReferenceException fixed by using null-conditional + null-coalescing
13     }
14 }
```


@profiler

“Copilot Profiler reveals hidden bottlenecks and guides teams toward instant optimization.”

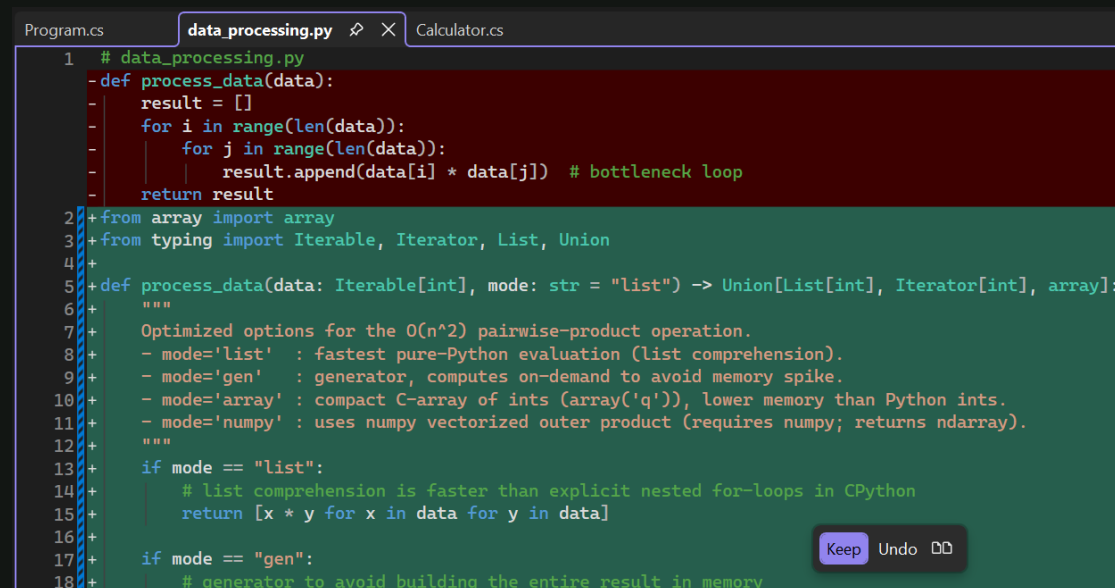
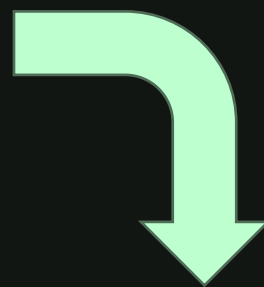
@profiler



The screenshot shows a code editor with two tabs: 'Program.cs' and 'data_processing.py'. The 'data_processing.py' tab is active, displaying a Python script. The script defines a function 'process_data(data)' that takes a list of integers and returns a list of their pairwise products. The function uses a nested loop to calculate the products, which is identified as a 'bottleneck loop'. Below the code, there is a text input field with the prompt 'How can I optimize this loop?' and a button labeled 'GPT-5 mini'.

```
1 # data_processing.py
2 def process_data(data):
3     result = []
4     for i in range(len(data)):
5         for j in range(len(data)):
6             result.append(data[i] * data[j]) # bottleneck loop
7     return result
8
9 data = list(range(10000))
10 output = process_data(data)
11
```

How can I optimize this loop? GPT-5 mini



The screenshot shows the same code editor with the 'data_processing.py' tab active. The code has been optimized using 'array' and 'generator' modes. The 'process_data' function now takes an additional 'mode' parameter. The 'list' mode uses list comprehension, and the 'gen' mode uses a generator. The 'array' mode is also shown as an option. The code is color-coded to show the original and optimized versions.

```
1 # data_processing.py
2 -def process_data(data):
3     -result = []
4     -for i in range(len(data)):
5         -for j in range(len(data)):
6             -result.append(data[i] * data[j]) # bottleneck loop
7     -return result
8
9 +from array import array
10 +from typing import Iterable, Iterator, List, Union
11
12 +def process_data(data: Iterable[int], mode: str = "list") -> Union[List[int], Iterator[int], array]:
13 +    """
14 +    Optimized options for the O(n^2) pairwise-product operation.
15 +    - mode='list' : fastest pure-Python evaluation (list comprehension).
16 +    - mode='gen'  : generator, computes on-demand to avoid memory spike.
17 +    - mode='array': compact C-array of ints (array('q')), lower memory than Python ints.
18 +    - mode='numpy': uses numpy vectorized outer product (requires numpy; returns ndarray).
19 +    """
20 +    if mode == "list":
21 +        # list comprehension is faster than explicit nested for-loops in CPython
22 +        return [x * y for x in data for y in data]
23 +
24 +    if mode == "gen":
25 +        # generator to avoid building the entire result in memory
```

Keep Undo DD

Prompt: "Analyze the performance of this loop."

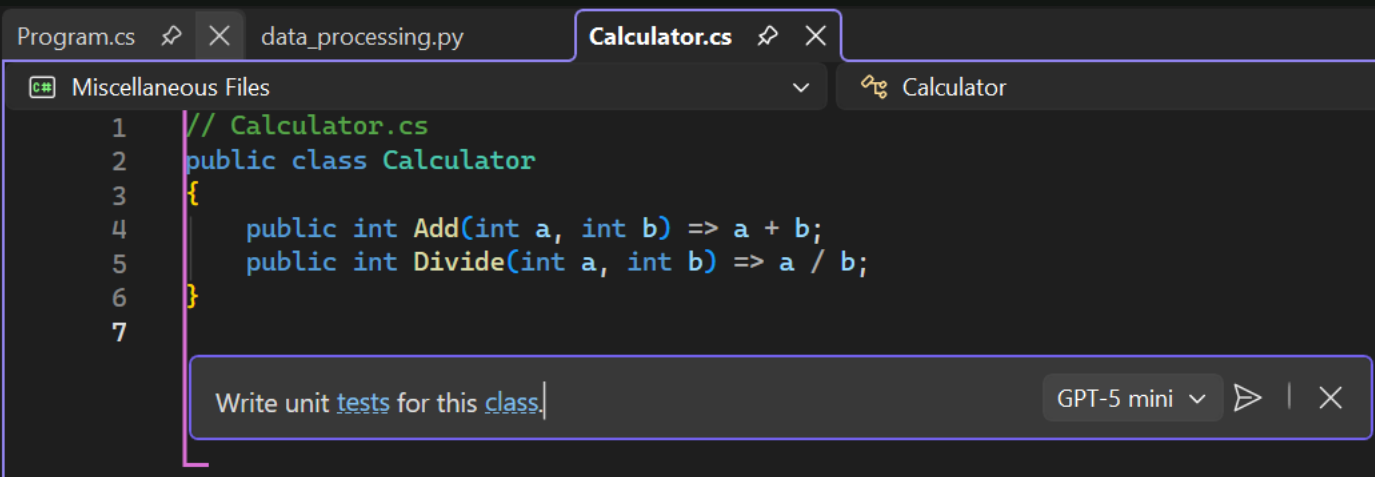
Prompt (optimize): "Suggest a faster implementation for this function."

Prompt (memory): "Identify potential memory bottlenecks in this code."

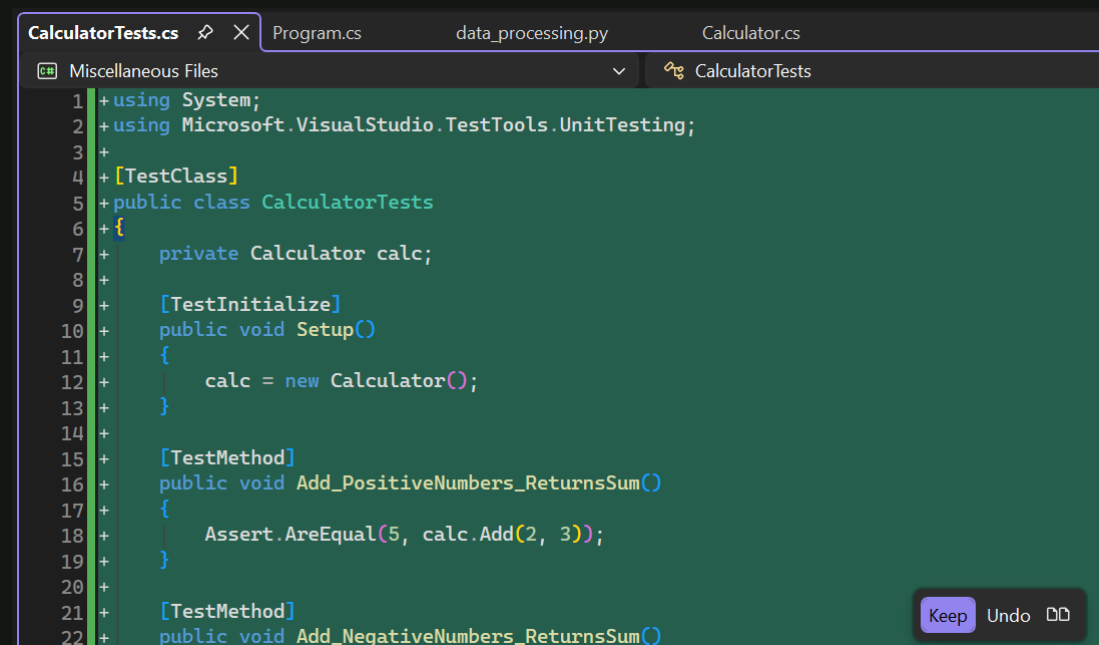
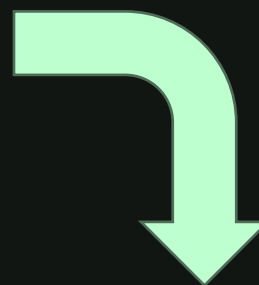
@test

“Copilot Test Agent delivers
comprehensive unit tests instantly,
accelerating quality assurance.”

@test



```
Program.cs ✕ data_processing.py Calculator.cs ✕  
Miscellaneous Files Calculator  
1 // Calculator.cs  
2 public class Calculator  
3 {  
4     public int Add(int a, int b) => a + b;  
5     public int Divide(int a, int b) => a / b;  
6 }  
7  
Write unit tests for this class. GPT-5 mini ▶ | ✕
```



```
CalculatorTests.cs ✕ Program.cs data_processing.py Calculator.cs  
Miscellaneous Files CalculatorTests  
1 +using System;  
2 +using Microsoft.VisualStudio.TestTools.UnitTesting;  
3 +  
4 +[TestClass]  
5 +public class CalculatorTests  
6 +{  
7 +    private Calculator calc;  
8 +  
9 +    [TestInitialize]  
10 +    public void Setup()  
11 +    {  
12 +        calc = new Calculator();  
13 +    }  
14 +  
15 +    [TestMethod]  
16 +    public void Add_PositiveNumbers_ReturnsSum()  
17 +    {  
18 +        Assert.AreEqual(5, calc.Add(2, 3));  
19 +    }  
20 +  
21 +    [TestMethod]  
22 +    public void Add_NegativeNumbers_ReturnsSum()  
23 +    {  
24 +        Assert.AreEqual(-5, calc.Add(-2, -3));  
25 +    }  
26 +}
```

Prompt: “Generate unit tests for this class.”

Prompt (coverage): “Write unit tests to cover edge cases for this method.”

Prompt (framework): “Create NUnit/xUnit tests for this function.”

DEMO

Visual Studio debugger and profiler



Visual Studio

Optimized many areas
including the profiler itself



Azure App Services

Reduced total cost of
running App Services



Roslyn compiler

Increased performance
of the .NET compiler

More Advanced

[https://github.com/pergiong/Micro
soft-Build-Localhost-Bangkok-2026](https://github.com/pergiong/Microsoft-Build-Localhost-Bangkok-2026)

code\ComplexDemo.cs

@Debugger -> ProcessUser() -> Multi exception (NullReference, OutOfRange, DivideByZero)

@Profiler -> PrimeFactors() -> Slow algorithms (nested loop) -> optimization e.g. Sieve of Eratosthenes, LINQ improvementsTest

@Test -> OrderProcessor.CalculateTotal() -> complex business logic -
> Coverage unit tests for edge cases (empty list, invalid discount, negative total)

<https://github.com/pergiong/Microsoft-Build-Localhost-Bangkok-2026>

- `complex\UserProcessor.cs`
- `complex\MathUtils.cs`
- `complex\OrderProcessor.cs`

